

**A
PROJECT REPORT**

On

AWS Movie Ticket System

Project Id: 25-PJ-BCA-17

Submitted In Partial Fulfillment of the Requirement

**For the Degree of
Bachelor of Computer Application**

By

Mayank Chauhan (231249000223)

**Under the Supervision of
Mr. Rajesh Gupta**



INDERPRASTHA ENGINEERING COLLEGE

GHAZIABAD, UTTAR PRADESH

AFFILIATED TO Chaudhary Charan Singh University, Meerut

(FORMERLY, MEERUT UNIVERSITY) | NAAC A++ ACCREDITED

(SESSION: 2025 - 2026)

DECLARATION

I Mayank Chauhan hereby declare that this submission is our own work and that, to the best of our knowledge and belief, it contains no material previously published or written by another person nor material which to a substantial extent has been accepted for the award of any other degree of the university or other institute of higher learning, except where due acknowledgment has been made in the text.

CERTIFICATE

This is to certify that Project Report entitled “AWS Movie Ticket System” which is submitted by Mayank Chauhan in partial fulfillment of the requirement for the award of degree BCA in Department of Computer Applications of Chaudhary Charan Singh University, Meerut is a record of the candidate own work carried out by him under my supervision. The matter embodied in this thesis is original and has not been submitted for the award of any other degree.

Date:

SUPERVISOR

ACKNOWLEDGEMENT

It gives us a great sense of pleasure to present the report of the BCA Project undertaken during BCA Final Year. We owe special debt of gratitude to Mr. Rajesh Gupta for his constant support and guidance throughout the course of our work. His sincerity, thoroughness and perseverance have been a constant source of inspiration for us. It is only his cognizant efforts that our endeavors have seen light of the day.

We also take the opportunity to acknowledge the contribution of Ms. Rajesh for her full support and assistance during the development of the project.

We also do not like to miss the opportunity to acknowledge the contribution of all faculty members of the department for their kind assistance and cooperation during the development of our project. Last but not the least, we acknowledge our friends for their contribution in the completion of the project.

ABSTRACT

*The AWS Movie Ticket System is a comprehensive web application designed to simplify and automate the process of browsing movies, viewing details, and booking tickets through a centralized digital platform. Developed using **Flask, HTML, CSS, and JavaScript**, with **AWS DynamoDB** as the database and **AWS SES/SNS** for communication services, the system ensures secure user management and real-time data handling.*

*Key modules include **user signup and login, movie listing and detail pages, and ticket booking**, each integrated with session management and cloud-based notifications. The application incorporates validation and exception handling to maintain data integrity and provide a smooth user experience. By replacing manual workflows with automated processes, the Movie Booking Management System reduces errors, improves efficiency, and enhances overall service delivery.*

*This project demonstrates the feasibility of deploying a **database-driven, cloud-integrated application** in a real-world entertainment context. It serves as a foundational model for future enhancements such as mobile app integration, personalized recommendations, and advanced analytics.*

Contents

DECLARATION	ii
CERTIFICATE	iii
ACKNOWLEDGEMENT	iv
ABSTRACT	v
LIST OF ABBREVIATIONS	xii

1. INTRODUCTION

1.1. Overview of the Project	1
1.2. Problem Statement	2
1.3. Project Objectives	3
1.4. Scope of the Project	4 - 5
1.5. Scope of the Project	5 - 6

2. THEORETICAL BACKGROUND

2.1 Introduction	7
2.2 AWS Movie System Architecture	7

3. SYSTEM DESIGN & DIGRAMS

3.1 System Architecture	8
3.2 Entity Relationship Digram(ERD)	9
3.3 Data Flow Digram	10-12
3.4 Database Schema	13-15

4. IMPELMANTATION & CODE STRUCTURE

4.1 Overview of Implementation	16
4.2 Implementation code	17-27

5. TESTING

5.1. Introduction to Testing	28
5.2. Testing Strategies	28
5.2.1 White Box Testing	28
5.2.2 Black Box Testing	29
5.3 Test Case	29-30

6. USER & ADMIN MANUAL

6.1. Overview	31
6.2. Getting Started (System Requirements)	31-32

6.3. Get Start with User	32-33
--------------------------------	-------

6.4. Admin Authentication	33
---------------------------------	----

7. INSTALLATION GUIDE

7.1. Prerequisites	34
--------------------------	----

7.2. Step 1: Install Python & Dependencies	34
--	----

7.3. Step 2: Database Setup	34
-----------------------------------	----

7.4. Step 3: Congigure AWS & SES	34
--	----

7.5. Step 4: Run the Web Application	35
--	----

8. RESULT & OUTPUT

8.1. Overview	31 – 42
---------------------	---------

8.2. System Screenshoys	36-42
-------------------------------	-------

9. CONCLUSION AND FUTURE WORK

9.1. Conclusion	43
-----------------------	----

9.2. Future Work	44
------------------------	----

10. REFERENCE & BIBLIOGRAPHY

List of Tables

3.1 Database Schema - user Table	13
3.2 Database Schema - admin Table	13
3.3 Database Schema - contactForm Table	14
3.4 Database Schema - moviesdata Table	14
3.5 Database Schema - bookings Table	15
5.1 System Test Cases	29-30

List of Figures

3.1 Entity Relationship Diagram (ERD).....	9
3.2 Data Flow Digram	10-12
8.1 Home Page	36
8.2 Sign up Page.....	37
8.3 Login Page	37
8.4 Forgot Page	38
8.5 Reset Page	38
8.6 About Ticket	39
8.7 Contact	40
8.8 Booking Page	40
8.9 Admin Dashboard	41-42

LIST OF ABBREVIATIONS

BCA	Bachelor of Computer Applications
CSS	Cascading Style Sheets
HTML	HyperText Markup Language
HTTP	Hypertext Transfer Protocol
JS	Java Script
AWS	Amazon Web Services
EC2	Elastic Compute Cloud
Dynamodb	No SQL Database
SNS	Simple Notification Service
SES	Simple Email Service
IAM	Identity & Access Management
ER	Entity Relationship
DFD	Data Flow Diagram
Python flask	Backend Integration
SaaS	Software as a Service
IaaS	Infrastructure as a Service
PaaS	Platform as a Service
API	Application Programming Interface

Chapter 1

INTRODUCTION

1.1 Overview of the Project

Our project is a cloud-based Movie Ticket Booking System developed using modern web technologies and deployed on Amazon Web Services (AWS). The system allows users to browse movies, book tickets, manage bookings, and contact the administrator through an easy-to-use web interface.

The front-end of the project is developed using HTML, CSS, and JavaScript, which provides an interactive and responsive user experience. The back-end is developed using Python Flask, which handles user requests, booking operations, authentication, and communication with the database.

This AWS-based system demonstrates the practical implementation of cloud computing concepts, database management, web development, and secure deployment techniques. The project is designed to provide a reliable, scalable, and efficient online movie ticket booking platform while giving hands-on experience with AWS cloud services and modern application architecture.

1.2 Problem Statement

1. Existing System Issues

In the traditional movie ticket booking system, users often face problems such as long queues at cinema halls, limited seat availability information, and manual booking processes. These methods are time-consuming, inefficient, and prone to human errors.

2. Lack of Real-Time Information

The existing system does not provide real-time updates on seat availability and movie schedules, making it difficult for users to plan and book tickets effectively.

3. Limited Accessibility:

Traditional booking methods require users to be physically present at the theatre which reduces convenience and accessibility, especially for users with time constraints.

4. Scalability and Performance Issues

Many existing systems are not capable of handling high user traffic during peak hours leading to slow performance or system failure.

5. Need for Cloud-Based Solution

There is a need for a modern, scalable, and secure system that can handle multiple users simultaneously and provide seamless performance. Cloud technology, particularly AWS, offers the required infrastructure to solve these issues.

6. Proposed Solution

To overcome the above problems, the proposed project aims to develop a cloud-based movie ticket booking system using AWS services. The system will provide features like online booking, real-time seat availability, secure data handling, and high scalability, thereby improving the overall user experience.

1.3 Project Objectives

1. Development Objective

To design and develop a cloud-based movie ticket booking system using AWS services for efficient and reliable performance.

2. User Functionality Objective

To provide users with features like movie browsing, show timings, seat selection, and online ticket booking in an easy and user-friendly manner.

3. Security Objective

To ensure secure user authentication and data protection using AWS security features and best practices.

4. Scalability Objective

To build a system that can handle multiple users simultaneously by utilizing the scalable infrastructure of AWS.

5. Performance Objective

To achieve fast response time and high availability through optimized cloud deployment.

1.4 Scope of the Project

1. Functional Scope

The project covers the development of a system that allows users to browse movies, Check show timings, select seats, and book tickets online through a web-based platform.

2. Technical Scope

The system is implemented using AWS cloud services such as EC2 for hosting, S3 for storage, DynamoDB for database management, ensuring scalability and reliability.

3. User Scope

The system is designed for general users (customers) who want to book movie tickets online easily and efficiently from any location.

4. Operational Scope

The platform manages user registration, login, booking details, and seat availability, providing real-time updates and smooth operation.

5. Performance Scope

The system is capable of handling multiple users simultaneously with fast response time and high availability using cloud infrastructure.

6. Security Scope

The project ensures basic data security and user authentication using AWS security features and controlled access.

7. Limitations of Scope

The project is limited to a basic implementation, and features like online payment gateway, mobile app, and advanced analytics are not included in the current version.

8. Future Scope

The system can be extended to include online payments, mobile application support, AI-based recommendations, and integration with real cinema databases.

1.5 Tools Used

The Web Application Project leverages a combination of modern frameworks, cloud services, and development utilities to deliver a secure and scalable solution. The following tools were integral to its design and implementation:

- i. **Flask (Python Framework):** Used for building the web application's backend, handling routing, form submissions, and rendering templates. Its lightweight nature and modular design made it ideal for rapid development and maintainability.
- ii. **Amazon DynamoDB (NoSQL Database):** Provided persistent storage for user credentials, tokens, and session data. DynamoDB's scalability and serverless architecture ensured reliable data handling without manual infrastructure management.
- iii. **AWS Simple Email Service (SES):** Enabled global email delivery for password reset links and account notifications. SES ensured secure, authenticated communication between the application and users.
- iv. **Werkzeug Security Utilities:** Used for hashing and verifying passwords, ensuring secure credential storage and authentication.
- v. **HTML, CSS, and Jinja Templates:** Implemented the user interface for signup, login, forgot password, and reset password pages. These tools ensured a consistent, responsive, and user-friendly design.
- vi. **UUID Library:** Generated unique tokens for password reset workflows, ensuring secure and tamper-proof recovery links.

vii. **Boto3 (AWS SDK for Python)**: Facilitated seamless integration with AWS services such as DynamoDB and SES, enabling cloud-native functionality directly from the Flask application.

viii. **Session Management (Flask)**: Provided secure handling of user sessions, enabling login persistence and logout functionality.

Chapter 2

THEORETICAL BACKGROUND

2.1 Introduction

This chapter provides a comprehensive overview of the theoretical concepts and technologies that underpin the development of AWS Movie Ticket System. Understanding these components is crucial for appreciating the architectural decisions made during the project's development.

2.2 AWS Movie Ticket Architecture

Movie Ticket System is a model for users to see available movie tickets, book tickets, and can see the available theatres or timings as per their convenient.

Chapter 3

SYSTEM DESIGN & DIAGRAMS

3.1 System Architecture

The project follows Layered Client-Server Architecture combined with MVC principles to ensure modularity, scalability, and maintainability.

1. **Presentation Layer (User Interface):** Built using **HTML, CSS, and JavaScript**, this layer handles all front-end interactions. Users can sign up, log in, browse movies, view details, and initiate bookings through a responsive interface.
2. **Business Logic Layer (Application Layer):** Implemented with **Flask (Python)**, this layer processes user requests such as authentication, movie detail retrieval, and ticket booking. It also manages session handling, integrates AWS services (SES for emails, SNS for admin notifications), and enforces validation and exception handling to ensure smooth operations.
3. **Database Layer (Data Access Layer):** Powered by **AWS DynamoDB**, this layer stores and retrieves user records, movie data, and booking information. It ensures real-time access to structured data and supports scalability for growing datasets.

3.2 Entity Relationship Diagram (ERD)

The ER Diagram visualizes the data relationships in the system. We have two primary entities: **Users** and **Files**. A User can have multiple

ER DIAGRAM – AWS MOVIE TICKET BOOKING SYSTEM

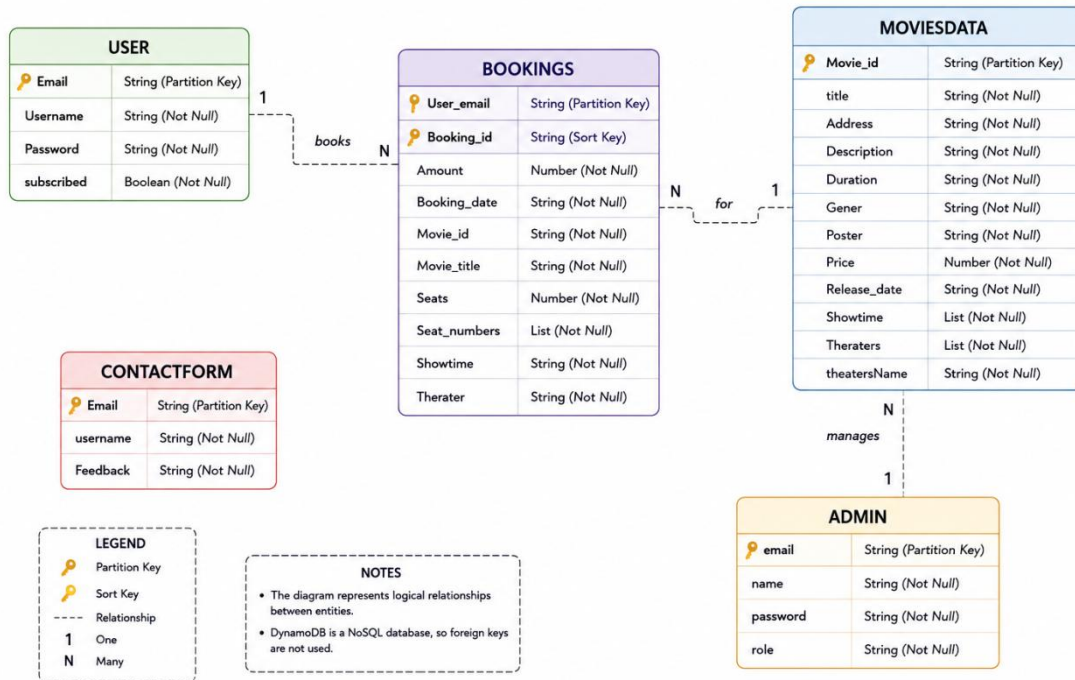


Figure 3.1: Entity Relationship Diagram (ERD)

3.3 Data Flow Diagram

The following ER Diagram represents the logical structure of the AWS Movie Ticket Booking System. It illustrates the entities involved in the application, including User, Admin, MoviesData, Bookings, and ContactForm, along with their attributes and logical relationships. The diagram demonstrates how users book movie tickets, how admins manage movie details, and how booking information is stored using Amazon DynamoDB..

LEVEL 0 DFD

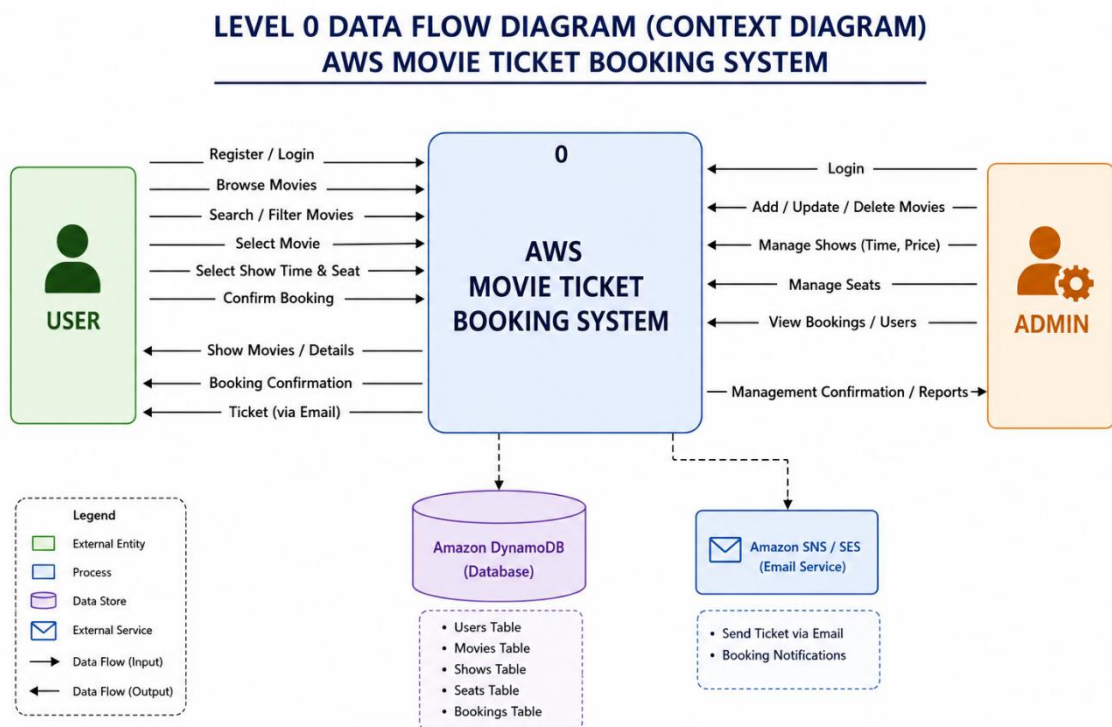
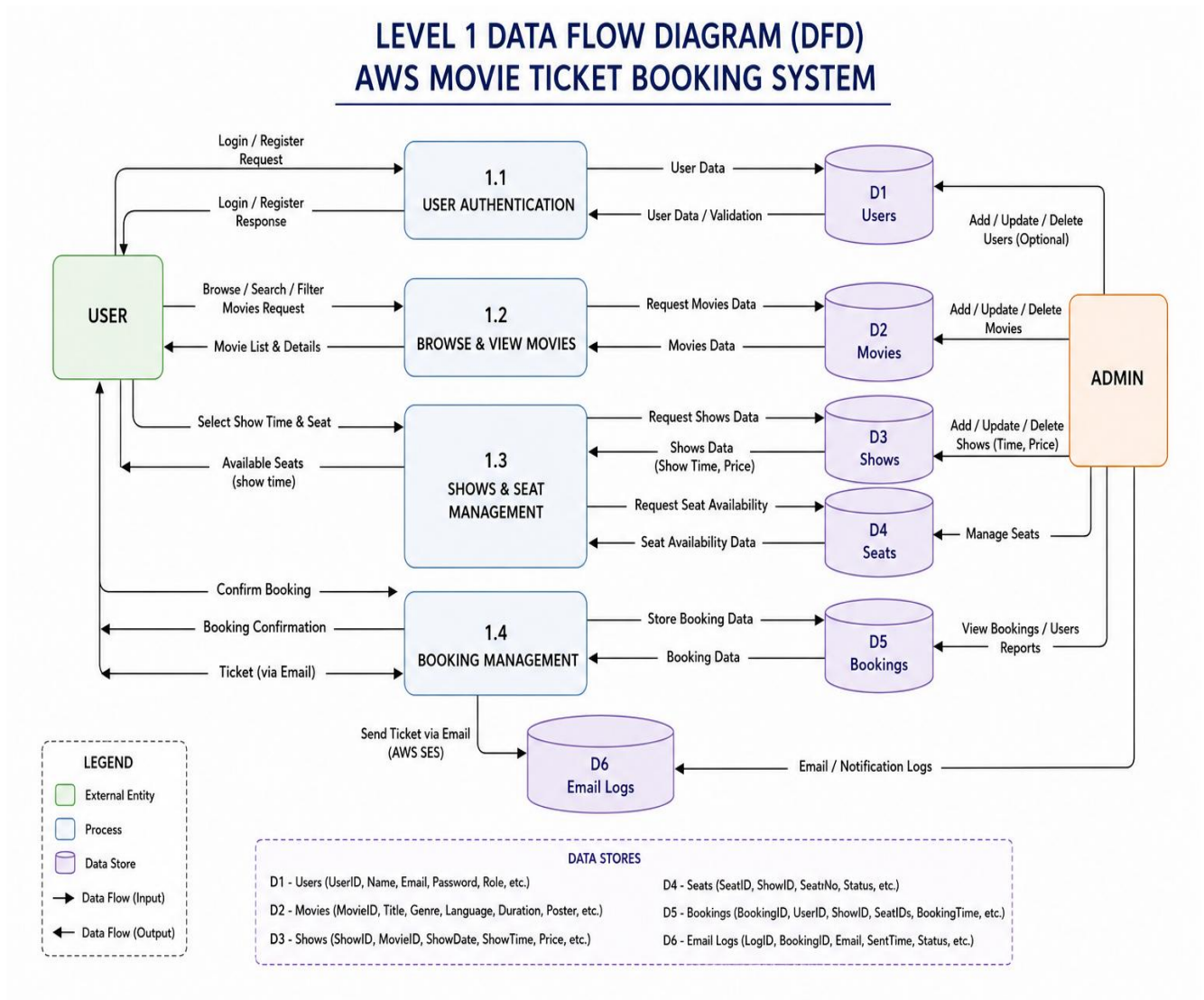


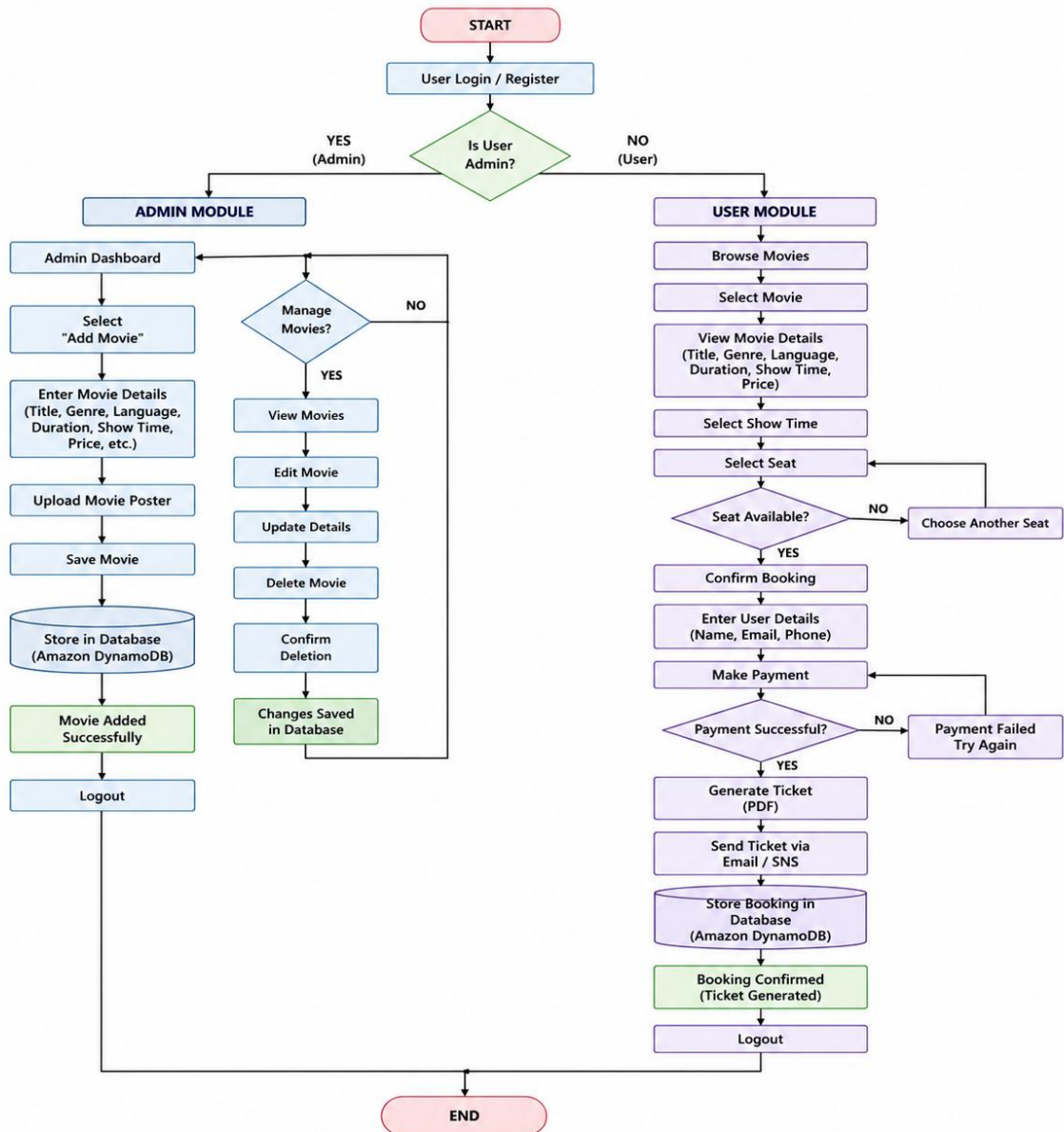
Figure 3.2 Data Flow diagram

LEVEL 1 DFD



LEVEL 2 DFD

LEVEL 2 FLOWCHART – AWS MOVIE TICKET BOOKING SYSTEM



3.4 Database Schema

The database tables are designed with strict constraints to ensure data integrity.

Table: User			
Field	Type	Constraints	Description
Username	String	Not Null	Display user name
Email	String	Partition Key	The display email of the user.
Password	String	Not Null	Store password in encrypted form.
subscribed	Boolean	Not Null	Check the user subscribed or not for promotional Newsletter / Marketing Emails

Table 3.1: Database Schema - user Table

Table : Admin			
Field	Type	Constraints	Description
name	String	Not Null	Display name of customer
email	String	Partition Key	Display nationality of customer
password	String	Not Null	Display phone number of customer
role	String	Not Null	Display post

Table 3.2 Database Schema – Admin Table

Table : ContactForm			
Field	Type	Constraints	Description
Email	String	Partition Key	Display sender email
Feedback	String	Not Null	Display feedback
username	String	Not Null	Display sender name

Table 3.3 Database Schema – ContactForm Table

Table : moviesdata			
Field	Type	Constraints	Description
Movie_id	String	Partition Key	Movie id for identify uniquely
Address	String	Not Null	Address of theaters
Description	String	Not Null	Description about movie
Duration	String	Not Null	How much movie is long
Genre	String	Not Null	Type of movie
Poster	String	Not Null	Poster
Price	Number	Not Null	Price
Release_date	String	Not Null	Date of movie release
Showtime	List	Not Null	Time duration on a day
Theaters	List	Not Null	Place of movie theater
theatersName	String	Not Null	Theaters name
title	String	Not Null	Movie name

Table 3.4 Database Schema – moviesData table

Table : bookings			
Field	Type	Constraints	Description
User_email	String	Partition Key	Movie id for identify uniquely
Booking_id	String	Sort Key	Address of theaters
Amount	Number	Not Null	Description about movie
Booking_date	String	Not Null	How much movie is long
Movie_id	String	Not Null	Type of movie
Movie_title	String	Not Null	Poster
Seats	Number	Not Null	Price
Seat_numbers	List	Not Null	Date of movie release
Showtime	String	Not Null	Time duration on a day
Theater	String	Not Null	Place of movie theater

Table 3.5 Database Schema – booking Table

Chapter 4

IMPLEMENTATION & CODE STRUCTURE

4.1 Overview of Implementation

This chapter provides a detailed breakdown of the source code used to build the **Movie Booking Management System**. The application is developed using **Flask (Python)** as the backend framework, combined with **HTML, CSS, and JavaScript** for the front-end interface. AWS services such as **DynamoDB, SES, and SNS** are integrated to handle database operations and communication services.

For better maintainability, the code logic has been modularized into the following components:

- **Imports:** Includes all necessary libraries such as Flask, boto3 (for AWS integration), and session management modules.
- **Models:** Defines the structure of entities like Users, Movies, Bookings, and ContactForm, ensuring consistent data handling.
- **Configuration:** Contains setup details for connecting to DynamoDB, configuring AWS SES/SNS, and managing application secrets.
- **Routes:** Implements the core functionality of the system, including signup, login, home page rendering, movie detail retrieval, and booking operations. Each route is designed with validation and exception handling to ensure smooth execution.

4.2 Implement code

Import all important Library and Packages

```
from flask import Flask, request, render_template, redirect, url_for
import os
from flask import jsonify
from datetime import datetime
from flask import session, flash
from werkzeug.utils import secure_filename
from datetime import datetime
import boto3
from boto3.dynamodb.conditions import Attr
from botocore.exceptions import ClientError
import uuid
from werkzeug.security import generate_password_hash, check_password_hash
```

App declaration, secret_key, session, upload_files and folder

```
app = Flask(__name__)
app.secret_key = os.environ.get("FLASK_SECRET_KEY")
app.config['SESSION_PERMANENT'] = False

app.config['UPLOAD_FOLDER'] = 'static/posters'
os.makedirs(app.config['UPLOAD_FOLDER'], exist_ok=True)
```

DynamoDB, SNS, SES Connection

```
dynamodb = boto3.resource('dynamodb', region_name='us-east-1')
sns = boto3.client('sns', region_name='us-east-1')
ses = boto3.client('ses', region_name='us-east-1')
def invite_customer_to_ses(email):
    try:
        # This triggers the AWS Verification email to the customer automatically
        ses.verify_email_identity(EmailAddress=email)
        print(f"Verification email sent to {email}")
    except Exception as e:
        print(f"SES Invite Error: {e}")

# dynamoddbtable(create these tables in dynamodb manually)
users_table = dynamodb.Table('users') # for store user data(login and signup)
```

```

admin_table = dynamodb.Table('admins') # for store and check admin login
contact_table = dynamodb.Table('ContactForm') # for feedback form
bookings_table = dynamodb.Table('bookings') # for track booked table
moviesdata_table = dynamodb.Table('moviesdata') # for store admin new movies

```

SNS, SES Logic

```

#=====
#SNS (Simple Notification Service)
#=====

```

```

def send_admin_notification(subject, message):
    """Send notification to admin topic (your email only)."""
    try:
        response = sns.publish(
            TopicArn=ADMIN_TOPIC_ARN,
            Subject=subject,
            Message=message
        )
        print(f"Admin notification sent! ID: {response['MessageId']}")
    except ClientError as e:
        print(f"Error sending admin notification: {e}")

```

```

#=====
#SES (Simple Email Service)
#=====

```

```

def send_customer_email(email, subject, message):
    """Send booking confirmation directly to one customer email using SES."""
    try:
        response = ses.send_email(
            Source="mayankchauhan.61z1@gmail.com", # sender (must be verified in
            # receiver (customer who booked)
            Destination={"ToAddresses": [email]},
            Message={
                'Subject': {'Data': subject},
                'Body': {'Text': {'Data': message}}
            }
        )
        print(f"Booking email sent to {email}! ID: {response['MessageId']}")
    except ClientError as e:
        print(f"Error sending customer email: {e}")

```

**Routes(/, /home, /contactForm, /about,
/search_suggestion, /admindashboard, /AdminLogin,
/movies/movie_id)**

SIMPLE ROUTE

```
@app.route('/')
def index():
    response = moviesdata_table.scan()
    db_movies = {m['movie_id']: m for m in response.get('Items', [])}

    # Merge: Code Movies + Database Movies
    all_movies = {**movies, **db_movies}
    return render_template('index.html', movies=movies, slides=slides)
```

HOME ROUTE

```
@app.route('/home')
def home():
    response = moviesdata_table.scan()
    db_movies = {m['movie_id']: m for m in response.get('Items', [])}
    all_movies = {**movies, **db_movies}
    if 'user' in session:
        return render_template('index.html', name=session['user'], movies=all_movies,
                               slides=slides)
    else:
        return render_template('index.html', name="Guest", movies=all_movies,
                               slides=slides)
```

ABOUT ROUTE

```
@app.route('/about')
def about():
    return render_template('about.html')
```

WHEN CHICK OR TYPE ON SERACHBAR IT ACTIVATE

```
@app.route('/search_suggestions')
def search_suggestions():
    query = request.args.get('query', "").lower()
```

```

# Combine movies and slides dictionaries
all_movies = {**movies, **slides}

# Filter by title containing query
results = []
for movie_id, movie in all_movies.items():
    if query in movie['title'].lower():
        results.append({
            "title": movie["title"],
            "genre": movie["genre"],
            "movie_id": movie_id
        })

return jsonify(results)

#ADMIN LOGIN PAGE ROUTE

@app.route('/AdminLogin')
def AdminLogin():
    return render_template('AdminL&S.html')

# Movie Route

@app.route('/movie/<movie_id>')
def movie_detail(movie_id):
    # 1. Check hardcoded movies first
    movie_data = movies.get(movie_id) or slides.get(movie_id)
    if not movie_data:
        try:
            response = moviesdata_table.get_item(Key={'movie_id': movie_id})
            movie_data = response.get('Item')
        except Exception as e:
            print(f"DynamoDB Error: {e}")
            return "Database connection error", 500
    if not movie_data:
        return "Movie not found", 404

    return render_template('movie.html', movie_data=movie_data,
movie_id=movie_id)

```

Login & Signup Route (/signup, /login)

```
#####
# SIGNUP API
#####

@app.route('/signup', methods=['GET', 'POST'])
def signup():
    if request.method == 'GET':
        return render_template('L&S.html')

    # POST signup
    name = request.form.get('name')
    email = request.form.get('email')
    password = request.form['password']

    hashed_password = generate_password_hash(password)

    # 1. Save to DynamoDB
    try:
        users_table.put_item(
            Item={
                'email': email,
                'name': name,
                'password': hashed_password,
                'subscribed': False
            },
            ConditionExpression='attribute_not_exists(email)'
        )
    except Exception as e:
        if "ConditionalCheckFailedException" in str(e):
            flash("An account with this email already exists. Please login.")
        else:
            print(f"DB Error: {e}")
            flash("Database error. Please try again.")
            return redirect(url_for('signup'))

    # 2. SES and SNS Notifications
    try:
        invite_customer_to_ses(email)

        confirmation_message = "user signup successfully! Please check your email to verify"
        send_customer_email(email, "Movie Ticket Booking Confirmation", confirmation_message)

        send_admin_notification("New User Signup", f"User {name} ({email}) has registered.")
    except Exception as e:
        print(f"Notification Error: {e}")

    # 3. Redirect to login
    return redirect(url_for('login'))
```



```

=====
# LOGIN API
=====
@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'GET':
        return render_template('L&S.html')

    # POST login
    email = request.form['email']
    password = request.form['password']

    response = users_table.get_item(Key={'email': email})
    user_item = response.get('Item')

    if user_item and check_password_hash(user_item['password'], password):
        username = user_item.get('name') or user_item.get('username')
        session['user'] = email
        session['email'] = email
        session['username'] = username

    try:
        confirmation_message = "You have successfully logged into ShowTime!"
        send_customer_email(email, "Login Notification", confirmation_message)
    except Exception as e:
        print(f"Login Email Error: {e}")

    flash("Login successful!", "success")
    return redirect(url_for('home'))
else:
    flash("Wrong email or password", "error")
    return redirect(url_for('login'))

```

Forgot & Reset Route(/forgot, /reset/<token>)

```

=====
#Forgot Password
=====

@app.route('/forgot', methods=['GET', 'POST'])
def forgot_password():
    if request.method == 'POST':
        email = request.form['email']
        response = users_table.get_item(Key={'email': email})
        user = response.get('Item')

    if not user:
        flash("Email not registered.")
        return redirect(url_for('forgot_password'))

    # Generate reset token and update DynamoDB
    reset_token = str(uuid.uuid4())
    users_table.update_item(
        Key={'email': email},
        UpdateExpression="set reset_token=:t",

```

```

        ExpressionAttributeValues={'t': reset_token}
    )

    # Build reset link
    reset_link = f"http://35.153.200.170/reset/{reset_token}"

    # Using your existing helper function!
    subject = "Password Reset - ShowTime"
    message = f"Click here to reset your password: {reset_link}"

    send_customer_email(email, subject, message)

    flash("Password reset link sent to your email.")
    return redirect(url_for('login'))

return render_template('forgot.html')

#=====
#Reset Password
#=====

@app.route('/reset/<token>', methods=['GET', 'POST'])
def reset_password(token):
    if request.method == 'POST':
        new_password = request.form['password']
        # Ensure your app uses the same hashing method as your signup
        hashed_pw = generate_password_hash(new_password)

        # Find user with this specific reset_token
        response = users_table.scan(
            FilterExpression=Attr('reset_token').eq(token)
        )
        items = response.get('Items', [])

        if not items:
            flash("Invalid or expired token.")
            return redirect(url_for('login'))

        email = items[0]['email']

        # Update password and CLEAN UP (remove the token so it can't be used again)
        users_table.update_item(
            Key={'email': email},
            UpdateExpression="set password=:p remove reset_token",
            ExpressionAttributeValues={'p': hashed_pw}
        )

        flash("Password updated successfully. Please log in.")
        return redirect(url_for('login'))

    # GET request → show reset page with the token passed to the form
    return render_template('reset.html', token=token)

```

Logout Routes(/logout)

```
#####  
# Logout  
#####  
  
@app.route('/logout')  
def logout():  
    # □ Clear session on logout  
    session.clear()  
    flash("Logged out successfully!", "info")  
    return redirect(url_for('home'))
```

Ticket Booking Routes(/book)

```
#####  
# Booking action  
#####  
  
@app.route('/book', methods=['POST'])  
def book():  
    if 'user' not in session:  
        return jsonify({"message": "You must login before booking tickets."}), 403  
  
    # Support JSON and Form data  
    data = request.get_json(silent=True) or request.form.to_dict()  
    customer_email = session.get('user')  
    movie_id = data.get('movie_id')  
  
    # 1. Fetch Movie Details (Code or DB)  
    movie = movies.get(movie_id) or slides.get(movie_id)  
    if not movie:  
        res = moviesdata_table.get_item(Key={'movie_id': movie_id})  
        movie = res.get('Item')  
  
    if not movie:  
        return jsonify({"message": "Movie not found."}), 400  
  
    # 2. Calculate values  
    seats = int(data.get('seats', 1))  
    total_amount = seats * movie.get('price', 0)  
    theater = data.get('theater', 'Default Theater')  
    showtime = data.get('showtime', 'Default Time')  
    seat_list = data.get('seat_numbers', [])  
    if isinstance(seat_list, str): seat_list = seat_list.split(',')  
  
    # 3. Create the Summary Message  
    confirmation_message = (  
        f"Booking Confirmed!\n\n"  
        f"Movie: {movie['title']}\n"  
        f"Theater: {theater}\n"  
        f"Showtime: {showtime}\n"  
        f"Seats: {seats} ({', '.join(seat_list)})\n"  
        f"Amount Paid: ₹ {total_amount}\n\n")
```

```

        f"Thank you for booking with us! "
        f"User: {customer_email}"
    )

    try:
        # A. Save to DynamoDB
        bookings_table.put_item(Item={
            'user_email': customer_email,
            'booking_id': str(uuid.uuid4()),
            'movie_id': movie_id,
            'movie_title': movie['title'],
            'seats': seats,
            'seat_numbers': seat_list,
            'amount': total_amount,
            'theater': theater,
            'showtime': showtime,
            'booking_date': datetime.utcnow().isoformat()
        })

        # B. Send SNS Admin Alert (So you know you made a sale!)
        try:
            send_admin_notification("New Ticket Sold!", confirmation_message)
        except Exception as e:
            print(f"SNS Error: {e}")

        # C. Send SES Customer Email
        try:
            send_customer_email(customer_email, f"Ticket: {movie['title']}",
confirmation_message)
        except Exception as e:
            print(f"SES Error: {e}")

        return jsonify({"message": f"Booking successful for {movie['title']}!"})

    except Exception as e:
        print(f"Critical Booking Error: {e}")
        return jsonify({"error": "Database error. Please try again."}), 500

```

Booked Ticket

Route(/booked_seats/<movie_id>/<showtime>)

```

#=====
# already booked tickets
#=====

@app.route('/booked_seats/<movie_id>/<showtime>', methods=['GET'])
def booked_seats(movie_id, showtime):
    response = bookings_table.scan(
        FilterExpression=Attr('movie_id').eq(movie_id) &
Attr('showtime').eq(showtime)
    )
    booked = []

```

```

for item in response['Items']:
    booked.extend(item['seat_numbers'])
return jsonify({"booked_seats": booked})

```

Admin Login Route()

```

#=====
# Admin Login
#=====

@app.route('/admin/login', methods=['GET', 'POST'])
def admin_login():
    if request.method == 'GET':
        return render_template('AdminL&S.html')

    # POST
    email = request.form['email']
    password = request.form['password']

    response = admin_table.get_item(Key={'email': email})

    if 'Item' not in response:
        return render_template('AdminL&S.html', msg="Admin not found")

    stored_password = response['Item']['password']

    # if check_password_hash(stored_password, password):
    #     return render_template('AdminDashbord.html', name=response['Item']['name'],
movies=movies)
    if check_password_hash(stored_password, password):
        session['role'] = 'admin'
        session['user'] = email
        return redirect(url_for('admin_dashboard'))
    else:
        return render_template('AdminL&S.html', msg="Wrong password")

```

Contact or Feedback Form Route(/contact)

```

#=====
# Contact Form
#=====

@app.route('/contact', methods=['GET', 'POST'])
def contact():
    if request.method == 'GET':
        return render_template('contact.html')

    # POST method
    username = request.form['username']
    email = request.form['email']
    feedback = request.form['feedback']

```

```

contact_table.put_item(
    Item={
        'email': email,      # Partition key
        'username': username,
        'feedback': feedback
    }
)

# Send SNS notification
send_admin_notification("New Contact Form Submission", f"{username}
({email}) submitted feedback: {feedback}")

msg = "Your feedback has been submitted successfully!"
return render_template('contact.html', msg=msg)

```

Admin Dashboard and movie delete

Route(/admin/dashboard, /delete/<movie_id>)

```

#=====
# Admin Dashbord (Add Movie)
#=====

@app.route('/admin/dashboard', methods=['GET', 'POST'])
def admin_dashboard():
    if session.get('role') != 'admin':
        return "Unauthorized", 403
    # Fetch from DB so they show up on login/refresh
    response = moviesdata_table.scan()
    db_movies = {m['movie_id']: m for m in response.get('Items', [])}
    # Merge hardcore local and dynamodb data
    all_movies = {**movies, **db_movies}
    # msg = None

    if request.method == 'POST':
        # movie_id = request.form['movie_id']
        movie_id = str(uuid.uuid4())
        title = request.form['title']
        description = request.form['description']
        genre = request.form['genre']
        release_date = request.form['release_date']
        price = int(request.form['price'])
        address = request.form['address']
        theatersName = request.form['theatersName']
        duration = request.form['duration']
        theaters = request.form['theaters'].split(',')
        showtimes = request.form['showtimes'].split(',')

        poster_file = request.files['poster']
        filename = secure_filename(poster_file.filename)
        poster_path = os.path.join(app.config['UPLOAD_FOLDER'], filename)
        poster_file.save(poster_path)

        movie_item = {

```

```

        'movie_id': movie_id,
        'title': request.form['title'],
        'description': request.form['description'],
        'genre': request.form['genre'],
        'release_date': request.form['release_date'],
        'price': int(request.form['price']),
        'address': request.form['address'],
        'theatersName': request.form['theatersName'],
        'duration': request.form['duration'],
        'theaters': request.form['theaters'].split(','),
        'showtimes': request.form['showtimes'].split(','),
        'poster': f'posters/{filename}"
    }

# 3. Save to DynamoDB
try:
    moviesdata_table.put_item(Item=movie_item)

    # Send notification
    send_admin_notification("New Movie Added", f"Movie
'{movie_item['title']}' added.")
    flash(f"Movie '{movie_item['title']}' added successfully!")

except Exception as e:
    print(f"Error saving to DynamoDB: {e}")
    flash("Error: Could not save movie data.")

return render_template('AdminDashbord.html', movies=all_movies)

#=====
# Delete Button Action
#=====

@app.route('/delete/<movie_id>', methods=['POST'])
def delete_movie(movie_id):
    if session.get('role') != 'admin':
        return "Unauthorized", 403
    if movie_id in movies:
        del movies[movie_id]
    # 2. Always attempt to delete from DynamoDB
    try:
        moviesdata_table.delete_item(Key={'movie_id': movie_id})
        flash("Movie deleted successfully!")
    except Exception as e:
        print(f"Database delete error: {e}")
        flash("Error: Could not delete from database.")

    return redirect(url_for('admin_dashboard'))

```

Port Route(For :5000)

```

if __name__ == '__main__':
    app.run(host="0.0.0.0", port=5000)

```

Chapter 5

TESTING

5.1 Introduction to Testing

Software testing is an investigation conducted to provide stakeholders with information about the quality of the software product or service under test. Software testing can also provide an objective, independent view of the software to allow the business to appreciate and understand the risks of software implementation.

5.2 Testing Strategies

5.2.1 White Box Testing

White box testing involves checking the internal code structure. In this project, we assured that:

- All loops in the Airline Management System terminate correctly.
- Database connections are closed properly even if errors occur.
- Exception handling is in place.

5.2.2 Black Box Testing

Black box testing involves checking functionality without seeing the code. We tested the application as a standard user:

- Logging in with incorrect passwords.
- Trying to access ‘/dashboard’ without logging in.
- Trying to upload invalid data of customers.

5.3 Test Cases

The following table documents the test cases executed to verify the system functionality.

Test ID	Test Description	Expected Result	Actual Result	Status
TC-01	Verify valid user login with correct credentials	User redirected to dashboard	Redirected successfully	Pass
TC-02	Verify invalid login	Error message displayed	Error displayed	Pass
TC-03	Verify empty login fields	Prompt to fill all required fields	Prompt shown	Pass
TC-04	Verify movie details page	Correct movie details displayed	Displayed successfully	Pass
TC-05	Verify movie booking process	Ticket booking completed successfully	Booking successful	Pass
TC-06	Verify invalid customer details	Prompt to enter correct information	Prompt shown	Pass
TC-07	Verify empty customer fields	Prompt to fill all required fields	Prompt shown	Pass
TC-08	Verify seat booking	Redirected to booking confirmation page	Redirected successfully	Pass
TC-09	Verify data stored in DynamoDB	Booking details saved in database	Data stored successfully	Pass
TC-10	Verify SNS notification service	Booking notification sent to user	Notification received	Pass
TC-11	Verify SES notification service	Email confirmation sent successfully to user	Email received successfully	Pass

TC-12	Verify unauthorized access restriction using IAM	Unauthorized access denied	Access denied	Pass
TC-13	Verify EC2 hosted application accessibility	Website loads successfully	Website accessible	Pass
TC-14	Verify admin can add movie details	Movie details added successfully	Added successfully	Pass
TC-15	Verify admin can delete movie details	Movie details deleted successfully	Deleted successfully	Pass
TC-16	Verify logout functionality	User logged out successfully	Logout successful	Pass
TC-17	Verify password validation	Password must contain at least 5 characters, symbols, or numbers	Validation performed successfully	Pass

Chapter 6

USER & ADMIN MANUAL

6.4 Overview

This user manual provides step-by-step instructions on how to use this AWS Movie Ticket System effectively. It designed for user and admin both.

6.2 Getting Started

6.2.1 Hardware Requirements

To ensure smooth development, testing, and deployment of the web application, the following hardware specifications are recommended:

- **Processor:** Intel Core i5 or higher (quad-core)
- **RAM:** Minimum 8 GB (16 GB recommended for cloud integration and concurrent testing)
- **Storage:** 256 GB SSD or higher for faster read/write operations
- **Network:** Stable broadband internet connection for AWS service integration
- **Additional:** Secure system with updated antivirus/firewall for safe cloud communication

6.2.2 Software Requirements

The project relies on a combination of programming frameworks, cloud services, and development tools:

- **Operating System:** Windows 10/11 or Linux (Ubuntu recommended for deployment)
- **Programming Language:** Python 3.x

- **Framework:** Flask (for backend routing, templating, and session management)
- **Database:** Amazon DynamoDB (NoSQL cloud database for user credentials and tokens)
- **Email Service:** AWS Simple Email Service (SES) for global password reset link delivery
- **Libraries:**
 1. Werkzeug (for password hashing and verification)
 2. Boto3 (AWS SDK for Python to connect with DynamoDB and SES)
 3. UUID (for generating unique reset tokens)
- **Frontend Tools:** HTML, CSS, Jinja templates for user interface design
- **IDE/Editor:** Visual Studio Code / PyCharm for development
- **Browser:** Google Chrome / Microsoft Edge for testing and debugging

6.3 Getting start with user

6.3.1 Accessing the Web Application

1. Ensure the internet running well.
2. Open the Web application through (<http://35.153.200.170/home>).
3. User can Browse movies without signup and login but for booking they have to signup or login through account there account.
4. User will then be able to access the AWS Movie Ticket System.

6.3.2 User Authentication

If user don't have account then they have to signup and login for full ticket booking.

6.3.3 Signup

1. Enter your Name.
2. Enter your email.
3. Enter your Password.
4. Click on "Register".

6.3.4 Login

1. Enter your registered email.
2. Enter your Password.
3. Click "Login".
4. Upon success, you will be redirected to the Home page.
5. Now you will be able to access all user features

6.4 Admin Authentication

6.4.1 Admin Login

1. Enter Admin email.
2. Enter Admin Password.

6.4.2 Adding Movies

1. On the Dashboard, look for the "Input Details" sections.
2. Click and fill the movies details.
3. Enter details of Movies like Movie title, Discription, Amount, Poster Image etc.
4. Click the "Add Movie" button.
5. A success message " Added Successfully!" will appear.
6. The Admin can confirm by checking in Dashboard.

6.4.3 Delete Movies

1. On the Dashboard, look for the "Details" section.
2. Click "Delete" Button to delete the movies from dashboard.

Chapter 7

INSTALLATION GUIDE

7.1 Prerequisites

Before installing and running the application, ensure your system meets the following requirements:

- **Operating System:** Windows 10/11, macOS, or Linux
- **Python:** Version 3.8 or higher
- **AWS Account:** Free tier or paid subscription, with access to DynamoDB and SES
- **IDE/Editor:** Visual Studio Code / PyCharm (recommended)
- **Browser:** Google Chrome / Microsoft Edge for testing

7.2 Step 1: Install Python and Dependencies

1. Download and install Python from <https://www.python.org/downloads/>.
2. Install required libraries using pip:

7.3 Step 2: Database Setup (DynamoDB)

1. Log in to your AWS Management Console.
2. Navigate to **DynamoDB** → **Create Table**.
3. Create a table named **Users** with:
 - **Primary Key:** `email` (String)
 - **Additional attributes:** `name`, `password`, `reset_token`
4. Save the table and confirm it is active.

7.4 Step 3: Configure AWS SES

1. In AWS SES, verify your sender email (e.g., `mayankchauhan.61z1@gmail.com`).

2. If SES is in sandbox mode, also verify recipient emails.
3. Request production access to send emails globally.

7.5 Step 4: Run the Application

1. Save your project files (`app.py`, templates, static CSS).
2. Open a terminal in your project directory.
3. Run the Flask app = `python3 app.py`
4. Open a browser and go to : <http://localhost:5000/>

Chapter 8

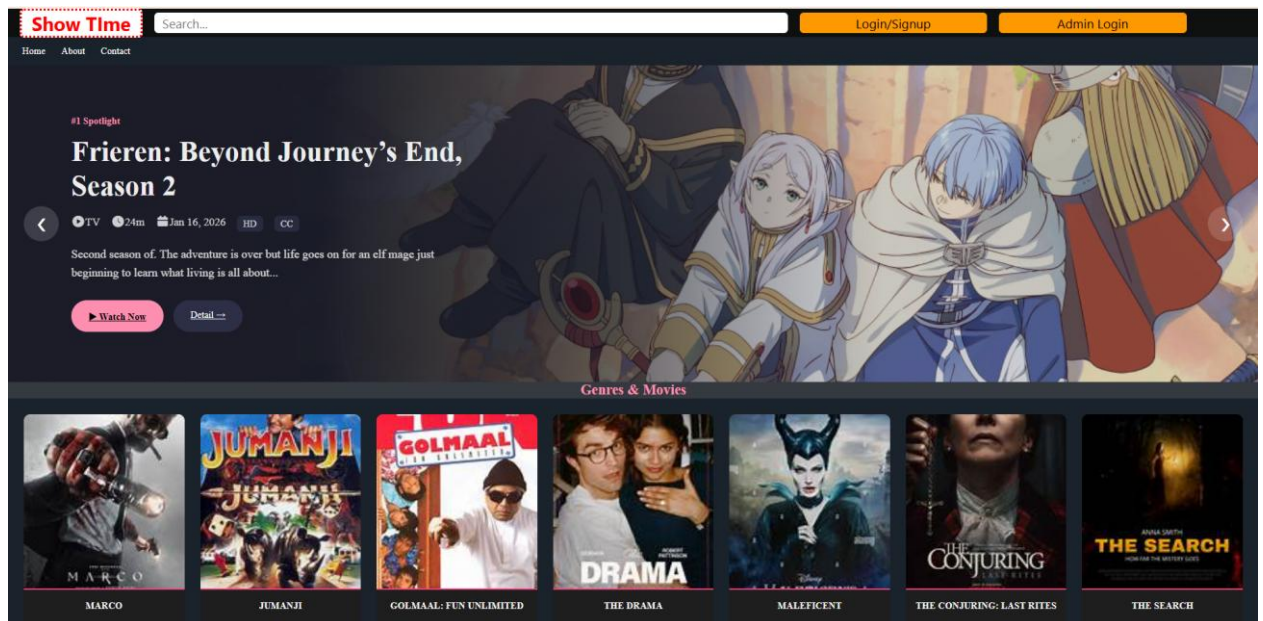
RESULTS AND OUTPUT

8.1 Overview

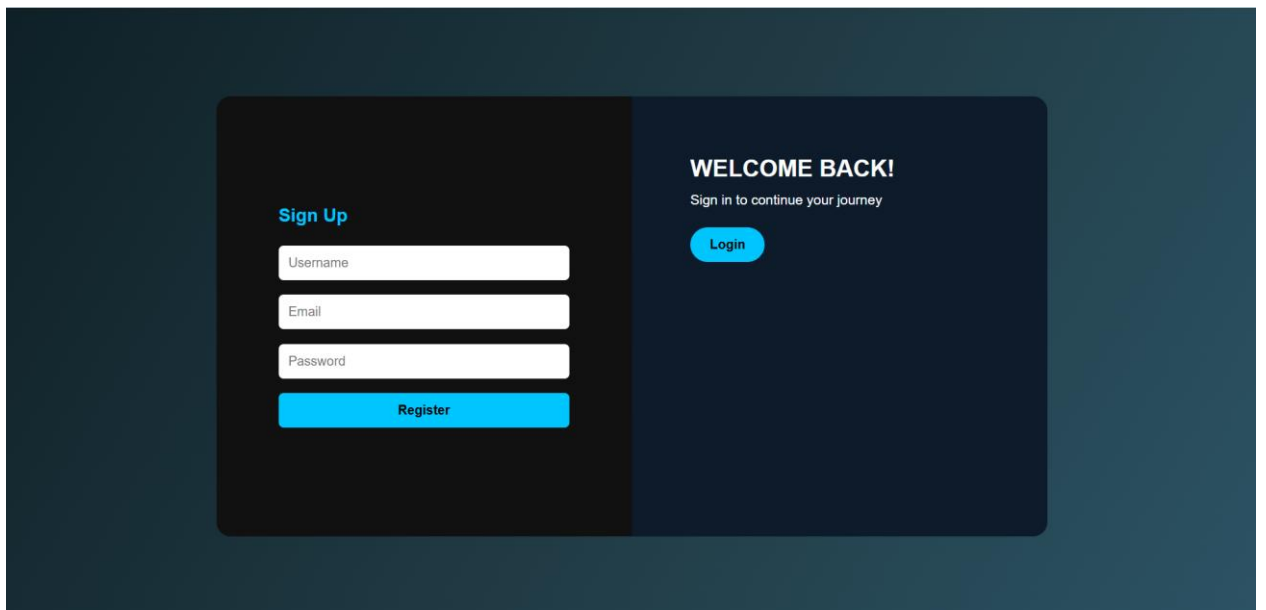
This chapter presents the visual output of Airline Management System. The user interface has been designed to be clean, minimalist, and easy to navigate.

8.2 System Screenshots

HOME PAGE



SIGN UP PAGE



The Sign Up page features a dark blue background. On the left, a dark navy box contains the 'Sign Up' heading in cyan, followed by three white input fields for 'Username', 'Email', and 'Password', and a cyan 'Register' button. On the right, a dark navy box contains the 'WELCOME BACK!' heading, a subtext 'Sign in to continue your journey', and a cyan 'Login' button.

Sign Up

Username

Email

Password

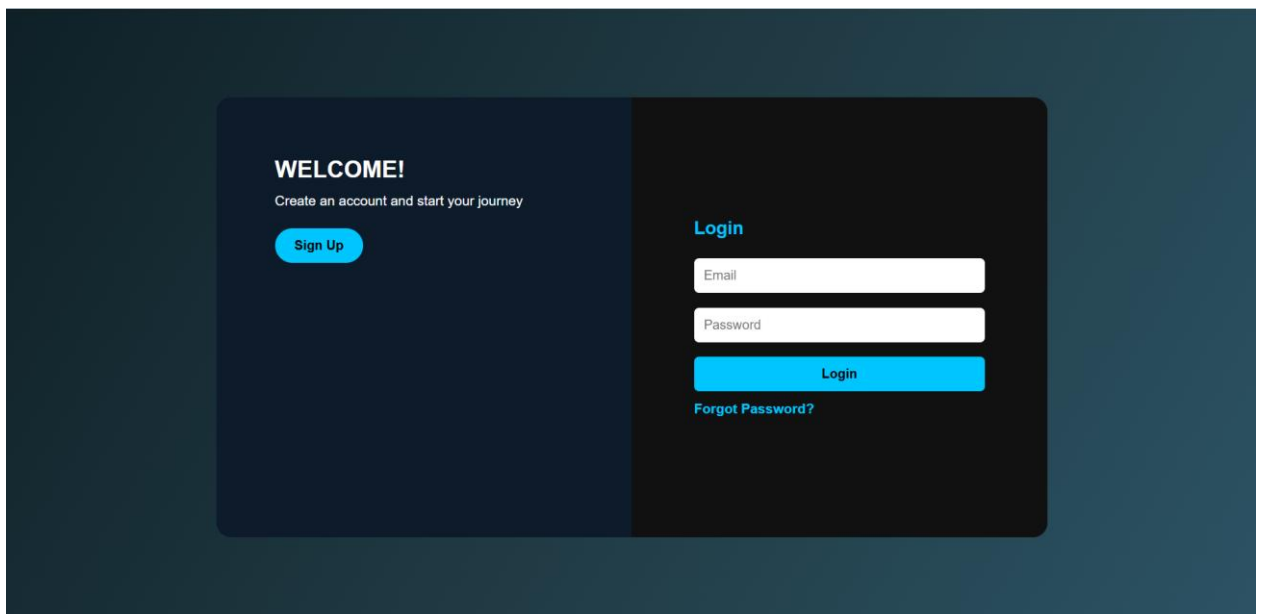
Register

WELCOME BACK!

Sign in to continue your journey

Login

LOGIN PAGE



The Login page features a dark blue background. On the left, a dark navy box contains the 'WELCOME!' heading, a subtext 'Create an account and start your journey', and a cyan 'Sign Up' button. On the right, a dark navy box contains the 'Login' heading in cyan, followed by two white input fields for 'Email' and 'Password', a cyan 'Login' button, and a 'Forgot Password?' link.

WELCOME!

Create an account and start your journey

Sign Up

Login

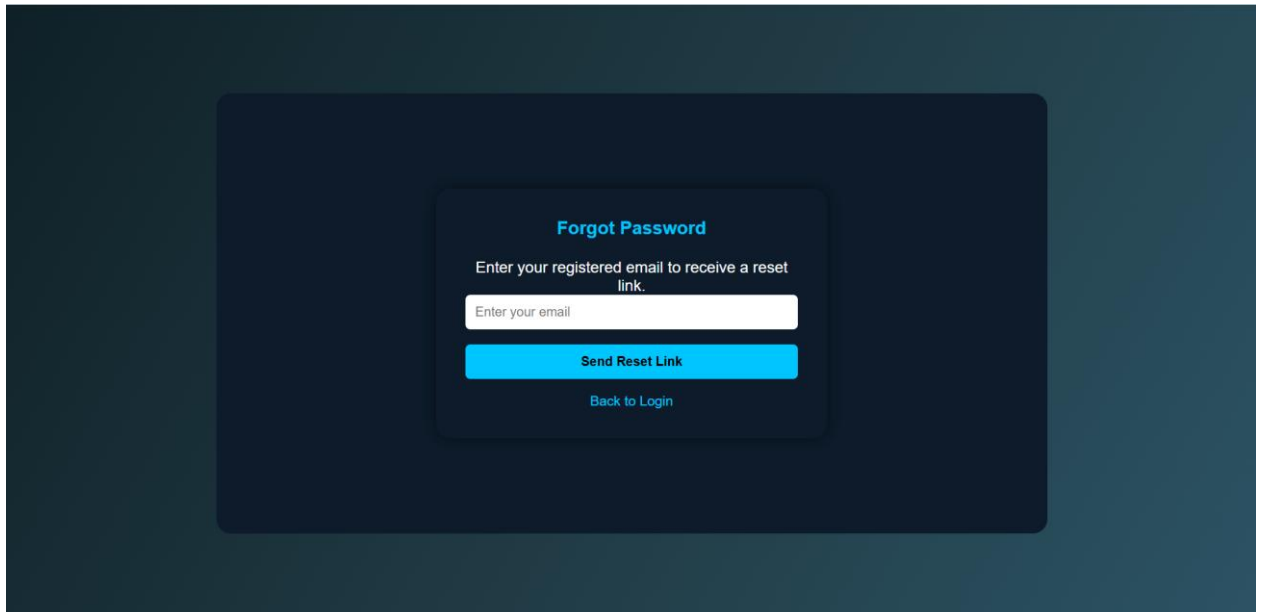
Email

Password

Login

[Forgot Password?](#)

FORGOT PASSWORD PAGE



The image shows a 'Forgot Password' page with a dark blue background. A central dark blue card contains the title 'Forgot Password' in cyan. Below it is the instruction 'Enter your registered email to receive a reset link.' in white. A white input field with the placeholder 'Enter your email' is present. Below the input field is a cyan button labeled 'Send Reset Link' in white. At the bottom of the card is a cyan link labeled 'Back to Login'.

Forgot Password

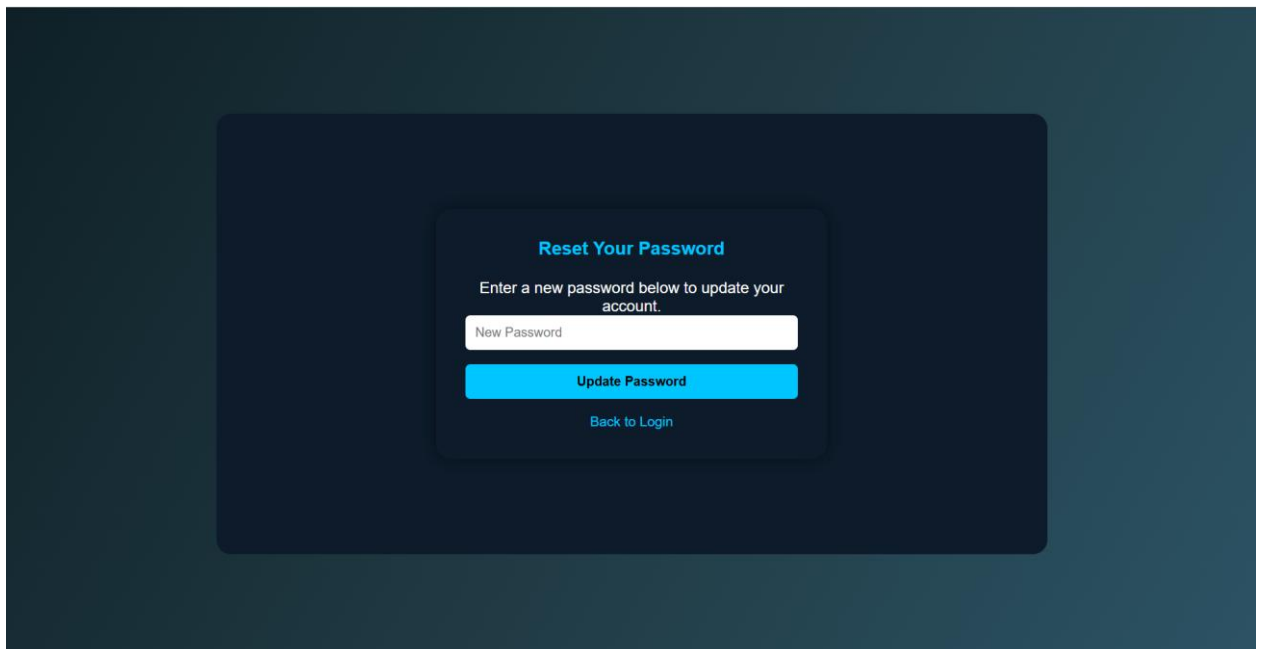
Enter your registered email to receive a reset link.

Enter your email

Send Reset Link

[Back to Login](#)

RESET PASSWORD PAGE



The image shows a 'Reset Your Password' page with a dark blue background. A central dark blue card contains the title 'Reset Your Password' in cyan. Below it is the instruction 'Enter a new password below to update your account.' in white. A white input field with the placeholder 'New Password' is present. Below the input field is a cyan button labeled 'Update Password' in white. At the bottom of the card is a cyan link labeled 'Back to Login'.

Reset Your Password

Enter a new password below to update your account.

New Password

Update Password

[Back to Login](#)

ABOUT PAGE

Show Time

Your Ultimate Online Movie Ticket Booking Platform

About Us

Show Time is a modern online movie ticket booking platform that helps users discover movies, view show timings, select seats, and book tickets quickly and securely—all in one place.

Why Choose MOVIE HUB?

 **Latest Movies**
Discover trending and upcoming movies instantly.

 **Smart Seat Selection**
Interactive seat layout for hassle-free booking.

 **Secure Payments**
Multiple payment options with full data protection.

Our Impact

1M+
Happy Users

5K+
Daily Bookings

300+
Cinema Partners

Our Mission

Our mission is to transform movie ticket booking into a smooth, enjoyable experience by blending technology, security, and entertainment.

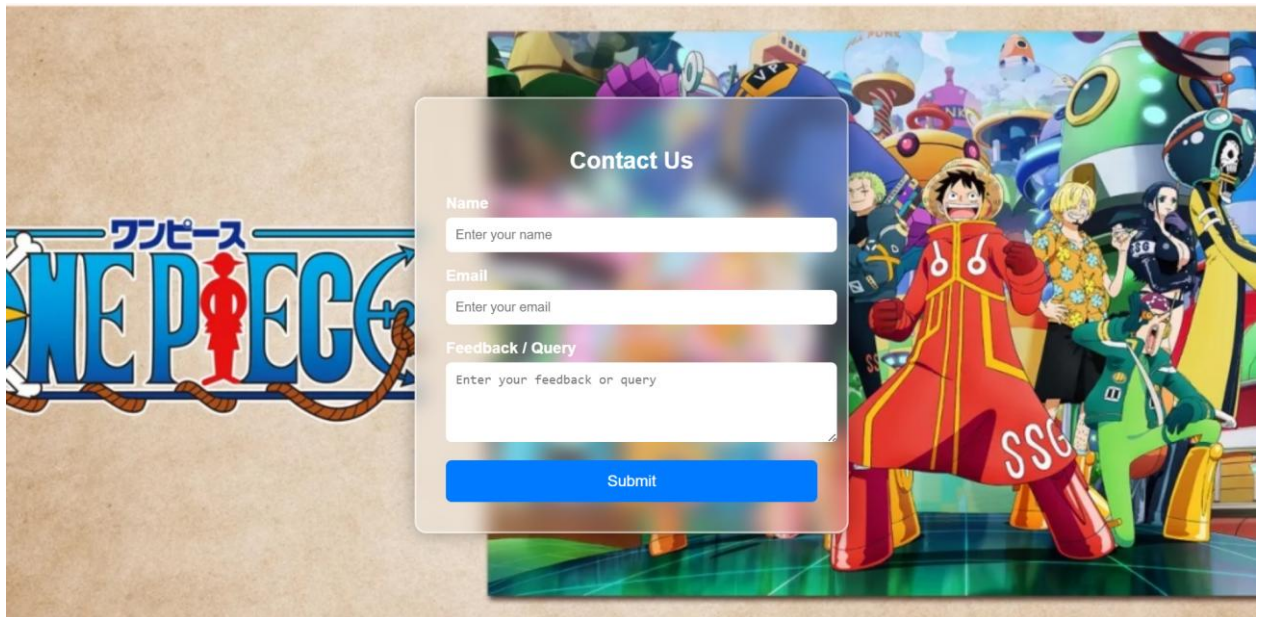
Ready for the Big Screen?

Book your next movie experience with Show Time today.

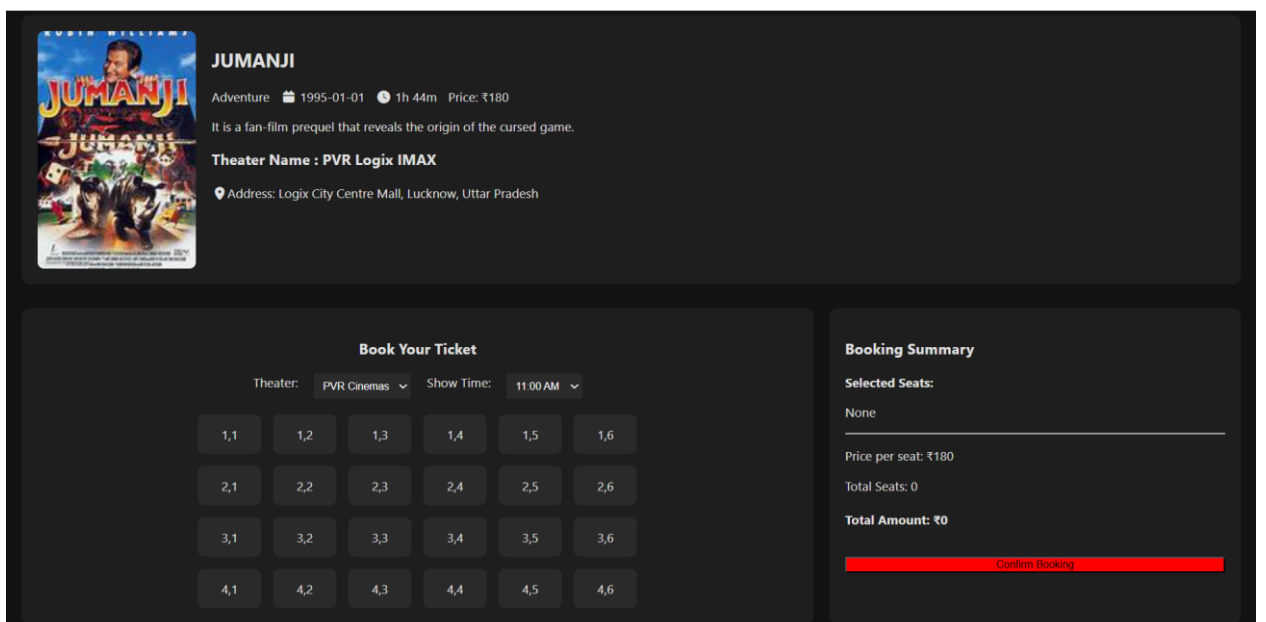
[Book Tickets Now](#)

© 2026 Show Time | All Rights Reserved

CONTACT PAGE



MOVIE BOOKING PAGE



ADMIN DASHBOARD

Admin Dashboard

 No file chosen

Current Movies

MARCO (MOV001)

It is a ruthless gangster seeking vengeance for his brother's brutal murder.

JUMANJI (MOV002)

It is a fan-film prequel that reveals the origin of the cursed game.

GOLMAAL: FUN UNLIMITED (MOV003)

It is about four mischievous, con-artist-Gopal,Madhav,Lucky & Laxman who are expelled from college.

THE DRAMA (MOV004)

This movie exploring intense human experiences,relationships and personal struggle.

MALEFICENT (MOV005)

This is a live-action Disney film that reimagines Sleeping Beauty from the villain's perspective, exploring her backstory as a betrayed fairy protector.

THE CONJURING: LAST RITES (MOV006)

It is the final entry in the main conjuring series, following Ed and Lorraine Warren as they face a definitive, high-stakes case based on the haunting of the Smurl family Pennsylvania.

THE SEARCH (MOV007)

The Search movie follows a man looking for life outside the universe who instead finds connection with a grieving family.

[Go Back to Home](#)

Current Movies

- **MARCO** (MOV001)
It is a ruthless gangster seeking vengeance for his brother's brutal murder.
- **JUMANJI** (MOV002)
It is a fan-film prequel that reveals the origin of the cursed game.
- **GOLMAAL: FUN UNLIMITED** (MOV003)
It is about four mischievous, con-artist-Gopal, Madhav, Lucky & Laxman who are expelled from college.
- **THE DRAMA** (MOV004)
This movie exploring intense human experiences, relationships and personal struggle.
- **MALEFICENT** (MOV005)
This is a live-action Disney film that reimagines Sleeping Beauty from the villain's perspective, exploring her backstory as a betrayed fairy protector.
- **THE CONJURING: LAST RITES** (MOV006)
It is the final entry in the main conjuring series, following Ed and Lorraine Warren as they face a definitive, high-stakes case based on the haunting of the Smurl family Pennsylvania.
- **THE SEARCH** (MOV007)
The Search movie follows a man looking for life outside the universe who instead finds connection with a grieving family.

[Go Back to Home](#)

Chapter 9

CONCLUSION AND FUTURE WORK

9.1 Conclusion

The Web Application Project successfully demonstrates the integration of Flask, DynamoDB, and AWS SES to deliver a secure, scalable, and user-friendly platform for account management. By automating critical workflows such as user registration, login authentication, and password recovery, the system replaces manual processes with a structured digital solution that enhances efficiency, accuracy, and overall user experience.

Through modular design and layered architecture, the project ensures maintainability, scalability, and robust data handling. Flask provides a flexible framework for routing and templating, DynamoDB offers reliable cloud-based storage for user credentials and session data, and AWS SES enables global communication through automated email delivery. Together, these technologies showcase practical application of modern web development, cloud computing, and secure authentication principles.

Beyond password recovery, the project highlights the importance of designing systems that balance usability with strong security measures. Features such as token-based workflows, session management, and hashed password storage establish a foundation for extending the application into a production-ready environment. Future enhancements may include token expiry policies, multi-factor authentication, domain-based email integration, and microservice-driven scaling, ensuring the system remains adaptable to evolving user and organizational needs.

In conclusion, the project demonstrates how cloud services and modern frameworks can be orchestrated to build a comprehensive, secure, and globally accessible web application — a solution that is both technically sound and practically valuable in today's digital ecosystem.

9.2 Future Scope

The Web Application Project, built using Flask, DynamoDB, and AWS SES, establishes a strong foundation for secure account management and global accessibility. However, to align with modern technological trends and evolving user expectations, the system can be enhanced in several impactful ways:

1. **Web-Based Expansion & UI Modernization:** Extend the current application with advanced front-end frameworks such as React or Angular to deliver a richer, more interactive user experience. This ensures platform independence, responsive design, and seamless deployment across devices.
2. **Mobile Application Development:** Create Android/iOS apps for user registration, login, and password recovery. Frameworks like Flutter or React Native can provide cross-platform compatibility, enabling users to manage accounts conveniently on mobile devices.
3. **Cloud-Native Scaling:** Host the application fully on cloud platforms such as AWS Elastic Beanstalk, EC2, or Kubernetes. This ensures scalability, high availability, and secure data storage, while integrating services like AWS CloudWatch for monitoring and auto-scaling.
4. **Advanced Security Features:** Implement token expiry policies, multi-factor authentication (MFA), and role-based access control. These measures strengthen account protection and align with industry standards for secure authentication.
5. **Artificial Intelligence Integration:** Use AI-driven analytics to detect suspicious login patterns, predict potential breaches, and personalize user experiences. Chatbots can be integrated for 24/7 support, guiding users through account recovery and troubleshooting.
6. **Domain & Email Deliverability Enhancements:** Acquire a custom domain and configure DNS records (SPF, DKIM, DMARC) to improve email deliverability and reduce spam risks. This ensures professional communication and trustworthiness in global deployments.
7. **Multilingual and Accessibility Support:** Expand usability by adding multilingual interfaces and accessibility features for differently abled users, ensuring inclusivity and compliance with accessibility standards.

Chapter 10

REFERENCE & BIBLIOGRAPHY

10.1 Training / Course Reference

- AWS Cloud Training Program.
- Platform: Skill Wallet.
- Topics Covered: Amazon EC2, Amazon SNS, AWS IAM, AWS SES, Amazon DynamoDB, Cloud Deployment, and Project Implementation of Movie Ticket Booking System.

10.2 Website (with urls)

- AWS Official Documentation
<https://docs.aws.amazon.com>
- Amazon EC2
<https://aws.amazon.com/ec2/>
- Amazon DynamoDB
<https://aws.amazon.com/dynamodb/>
- Amazon SNS
<https://aws.amazon.com/sns/>
- Amazon SES
<https://aws.amazon.com/ses/>
- AWS IAM
<https://aws.amazon.com/iam/>
- GeeksforGeeks
<https://www.geeksforgeeks.org/>
- W3Schools
<https://www.w3schools.com/>

